

Trabajo Práctico

Visualización de Datos

Asignatura	Visualización de Datos
Máster	Máster universitario en ingeniería y ciencia de datos
Curso	2024-2025

RESUMEN:

El documento describe un trabajo práctico sobre la visualización de datos enfocado en la **detección de amenazas de ciberseguridad en el tráfico de red de IoT**. Se detalla la creación de un *dashboard* interactivo utilizando **Streamlit y Altair** para **analizar comparativamente el tráfico normal y malicioso**, empleando el conjunto de datos UNSW-NB15. El proyecto aborda el preprocesamiento de datos, la exploración de visualizaciones y la integración de resultados parciales en una aplicación cohesiva, destacando la importancia de un diseño claro y una narrativa visual para profesionales de la ciberseguridad.

Tabla de Contenidos

Planteamiento del problema y objetivos de la visualización	3
Contexto del proyecto	3
Objetivo general	3
Subobjetivos y estructura del análisis	3
Visualización final integrada	4
Preparación de los datos (preprocesado)	5
Exploración y Preparación Inicial de Datos.....	5
Limpieza de datos	6
Filtrado y selección de variables	7
Transformación de variables	7
Procesado y análisis (estudios y visualizaciones parciales)	8
Paso 1: Experimentación con Visualizaciones Iniciales.....	8
Paso 2: Experimentación con las visualizaciones para la visión general	10
Paso 3: Experimentación con las visualizaciones para la Composición del Tráfico Malicioso.....	11
Paso 4: Análisis de los Efectos en el Rendimiento de la Red	12
Paso 5: Análisis del Impacto de cada Tipo de Ataque.....	13
Visualización (integración de los resultados parciales).....	15
Discusión, Conclusiones y posibles mejoras	18
Discusión y Conclusiones	18
Posibles mejoras y ampliaciones futuras	18
Anexo 1: Descripción de variables	20
Anexo 2: Instrucciones de despliegue del <i>dashboard</i>	22
Precondiciones	22
Ejecución del dashboard	22

Planteamiento del problema y objetivos de la visualización

Contexto del proyecto

En el contexto actual de creciente conectividad y expansión de dispositivos IoT (*Internet of Things*), las redes informáticas se enfrentan a un volumen cada vez mayor de amenazas de ciberseguridad. Uno de los grandes desafíos para profesionales del área es **detectar comportamientos anómalos o maliciosos** en el tráfico de red en tiempo real, para así poder mitigar posibles ataques antes de que afecten la integridad o disponibilidad de los sistemas.

Este trabajo se enmarca en la necesidad de construir herramientas visuales que ayuden a los analistas de ciberseguridad a interpretar grandes volúmenes de tráfico de red y a identificar patrones de ataque de manera más intuitiva y efectiva. Para ello, se ha desarrollado un *dashboard* interactivo utilizando Streamlit y Altair, y el conjunto de datos [UNSW-NB15](#), un dataset ampliamente utilizado para la investigación en detección de intrusiones.

El público destinatario de esta visualización son principalmente **profesionales o estudiantes de ciberseguridad**, así como analistas de datos que buscan explorar de forma visual y accesible las diferencias entre tráfico normal y malicioso, con el fin de entrenar modelos de detección o diseñar estrategias de protección.

Objetivo general

El objetivo principal de este dashboard es **visualizar y analizar comparativamente el tráfico de red normal y malicioso para identificar patrones** relevantes de ciberataques en entornos IoT.

Este objetivo se basa en el análisis de métricas clave como los bytes enviados y recibidos, los protocolos y servicios utilizados, las IPs implicadas y el tipo de ataque, con el fin de facilitar la comprensión de cómo se comporta el tráfico malicioso frente al tráfico legítimo.

Subobjetivos y estructura del análisis

Para alcanzar el objetivo global, se han definido los siguientes subobjetivos, cada uno desarrollado mediante visualizaciones parciales:

1. Describir el volumen y tipo de tráfico observado.

- a. Se analiza la cantidad total de conexiones y se presentan comparativas de tráfico normal frente a malicioso, destacando tendencias generales.

- b. Visualizaciones clave: conteo de conexiones, porcentaje de tráfico normal y malicioso, conteo de tipos de ataque, protocolos, servicios, IPs.

2. Comprender la composición del tráfico malicioso.

- a. Se examinan variables como protocolos, servicios, estados y puertos por tipo de ataque (DoS, Fuzzers, Exploits, etc.) mediante filtros y gráficos interactivos.
- b. Esto permite explorar los patrones que caracterizan cada tipología de ataque.
- c. Visualizaciones clave: gráficos de tarta donde se desglosa la composición de cada variable para cada tipo de ataque.

3. Comparar el rendimiento del tráfico normal vs malicioso

- a. Se analiza la distribución de bytes enviados y recibidos, así como la duración de la conexión frente a los paquetes enviados y recibidos, comparando los valores entre conexiones normales y maliciosas.
- b. Visualizaciones clave: diagrama de caja de bytes enviados, diagramas de dispersión de la duración y paquetes.

4. Analizar el impacto en la red por cada tipología de ataque

- a. Se estudian las principales IPs involucradas como fuente o destino de ataques, los paquetes enviados/recibidos y las tasas de pérdida en cada categoría de ataques frente.
- b. Visualizaciones clave: gráficos de barras que muestran, para cada dirección IP, la cantidad total de conexiones en las que participa, desglosada por categoría de ataque. Gráficos de barras que muestran el promedio del volumen de paquetes y número de paquetes perdidos por tipología de ataque.

Visualización final integrada

La integración de todas estas visualizaciones en un único *dashboard* interactivo permite obtener una visión completa del comportamiento del tráfico de red, con posibilidad de filtrar por tipo de tráfico, ataque o protocolo. Esto responde al objetivo global de apoyar visualmente el análisis de ciberseguridad, contribuyendo a una mejor comprensión de los riesgos presentes en redes IoT.

Las instrucciones detalladas para desplegar y ejecutar el *dashboard* localmente se encuentran disponibles en el Anexo 2 de esta memoria.

Preparación de los datos (preprocesado)

Para poder llevar a cabo un análisis eficaz del tráfico malicioso recogido en el conjunto de datos UNSW-NB15, ha sido necesario aplicar una serie de tareas de preprocesamiento. Este paso es fundamental para asegurar la calidad del análisis posterior y mejorar la eficiencia de los algoritmos utilizados. A continuación, se describen las principales acciones de preprocesamiento realizadas, enmarcadas dentro de las tres categorías clásicas del tratamiento previo de datos: limpieza de datos, selección/filtrado de variables, y transformación de variables.

Exploración y Preparación Inicial de Datos

El primer desafío fue entender a fondo el dataset UNSW-NB15, un conjunto complejo y amplio que contiene miles de conexiones etiquetadas como normales o maliciosas, y múltiples variables técnicas asociadas.

Exploración de los archivos:

El *dataset* combina tráfico real y ataques sintéticos, con más de 2.5 millones de registros y 49 características por registro, distribuidos en varios archivos CSV. Los archivos principales incluyen:

- **NUSW-NB15_features.csv**: Lista de las 49 características con su tipo.
- **UNSW-NB15_1.csv**, **UNSW-NB15_2.csv**, **UNSW-NB15_3.csv** y **UNSW-NB15_4.csv**: Archivos con datos de tráfico.
- **NUSW-NB15_GT.csv**: Archivo de *ground truth* que contiene etiquetas reales de los ataques.
- **UNSW-NB15_LIST_EVENTS.csv**: Resumen agregado de eventos por categoría y subcategoría de ataque.
- **Training and Testing Sets**: Particiones específicas para entrenamiento y prueba destinadas a modelos de Machine Learning.

Para esta visualización y análisis se utilizaron **NUSW-NB15_features.csv** y los archivos de datos **UNSW-NB15_1.csv**, **UNSW-NB15_2.csv**, **UNSW-NB15_3.csv** y **UNSW-NB15_4.csv**. Son los que contienen la información completa y detallada del tráfico normal y malicioso. Los demás son útiles para validar y contextualizar la información en el desarrollo de algoritmos de *Machine Learning*, pero no fueron necesarias para la construcción del *dashboard*.

Significado de las variables:

Antes de comenzar con el análisis, también realicé una exploración detallada del significado de cada variable presente en el dataset UNSW-NB15 para comprender mejor la información disponible y su relevancia (ver Anexo 1).

Cada registro del conjunto de datos representa un flujo de red, es decir, una conexión única entre un dispositivo de origen y otro de destino, ocurrida durante un intervalo temporal determinado.

Sesiones de inspección de datos en Jupyter Notebook:

Para la inspección inicial, utilicé **Jupyter Notebook** como entorno principal de exploración y experimentación. En esta fase inicial, dediqué varias sesiones a inspeccionar las variables disponibles, su distribución y calidad. Detecté valores faltantes, inconsistencias y variables poco relevantes, et. lo que me llevó a los siguientes pasos del preprocesado.

Limpieza de datos

Es una fase crítica en la preparación de cualquier conjunto de datos. En este caso, se han llevado a cabo las siguientes acciones:.

Conversión de tipos de datos: Se partió de un archivo auxiliar que contenía la definición de las características del conjunto de datos: el archivo `NUSW-NB15_features.csv`. A partir de este archivo se construyó un diccionario de tipos de datos (`dtype_dict`) que permitía asignar a cada columna su tipo correcto (por ejemplo, `float32`, `Int64`, `category`, etc.).

Corrección de valores erróneos: Se identificaron valores que no podían ser tratados directamente como enteros (por ejemplo, valores en formato hexadecimal como `'0x1A'` o entradas con guiones o espacios vacíos como `'-'` o `' '`). Se diseñó una función personalizada que convertía dichos valores en enteros.

Tratamiento de valores nulos: Aunque no se eliminaron explícitamente filas con valores nulos, sí se aplicó un control sobre las conversiones para que los campos no válidos fueran tratados como `NaN`, y se usaron métodos como `pd.to_numeric(..., errors='coerce')` para manejar conversiones que podrían fallar.

Limpieza de variables categóricas: Se identificaron columnas categóricas con valores inconsistentes por presencia de espacios en blanco o variaciones mínimas en la escritura. Se aplicó una limpieza de espacios (`str.strip()`) y se transformaron en variables de tipo `category`, lo que optimiza tanto el almacenamiento como el rendimiento computacional.

Corrección de etiquetas inconsistentes: En algunos casos, fue necesario estandarizar nombres de categorías. Por ejemplo, la categoría de ataque `'Backdoors'` se renombró como `'Backdoor'` para evitar duplicidades. Asimismo, valores como `'-'` en la columna `service` se sustituyeron por `'unknown'`, para mejorar la comprensión.

Filtrado y selección de variables

Dado que el conjunto completo contenía un elevado número de columnas, muchas de las cuales no eran relevantes para el análisis de tráfico malicioso, se aplicó un proceso de filtrado de variables para reducir el volumen de datos y enfocarse en aquellas variables clave para el estudio.

Selección de columnas relevantes:

Se seleccionaron explícitamente 20 columnas que contenían la información más relevante para la detección de patrones de comportamiento en el tráfico de red. Entre ellas se incluyen:

- Información de conexión: `proto`, `service`, `state`, `sport`, `dsport`.
- Identificación IP: `srcip`, `dstip`.
- Volumen de datos: `sbytes`, `dbytes`, `Spkts`, `Dpkts`, `sloss`, `dloss`, `dur`.
- Tiempo: `Stime`, `Ltime`.
- Variables auxiliares: `is_ftp_login`, `is_sm_ips_ports`.
- Etiquetas de análisis: `Label`, `attack_cat`.

Filtrado de instancias:

Algunos subobjetivos implicaron realizar un filtrado de instancias para centrarse exclusivamente en el tráfico malicioso. Para ello, se utilizó la columna `Label`, que indica si una conexión fue benigna (`Label = 0`) o maliciosa (`Label = 1`). Solo se conservaron las instancias donde `Label = 1`, ya que el análisis posterior se centraba en identificar orígenes, destinos y patrones de ataque dentro del tráfico malicioso.

Transformación de variables

Además de la limpieza y filtrado, se llevaron a cabo diversas transformaciones sobre los datos, tanto para mejorar la interpretabilidad como para facilitar su análisis temporal.

Conversión de marcas temporales: Las columnas `Stime` y `Ltime`, que representan el momento de inicio y final de cada conexión, se encontraban originalmente en formato timestamp Unix (segundos desde 1970-01-01). Estas variables fueron transformadas al tipo `datetime` mediante la función `pd.to_datetime(..., unit='s')`. Esta transformación es esencial para realizar análisis temporales, como detectar patrones por hora, día o semana.

Normalización de formatos: Además de la estandarización de nombres ya mencionada, se llevó a cabo una transformación general sobre las columnas categóricas, normalizando sus valores mediante métodos como `.str.strip()` y sustituyendo valores atípicos o inconsistentes.

Procesado y análisis (estudios y visualizaciones parciales)

El desarrollo de un *dashboard* interactivo para analizar el tráfico de red en entornos IoT, diferenciando entre tráfico normal y malicioso, representa un reto significativo debido a la complejidad y el volumen de datos involucrados. Este proyecto no solo implicó la creación de visualizaciones efectivas, sino también un proceso riguroso y detallado de exploración, limpieza y transformación de datos. La construcción del *dashboard* fue un camino **lleno de iteraciones, pruebas y ajustes constantes**, que permitieron refinar gradualmente cada aspecto para llegar a un producto final que satisface los objetivos planteados.

A continuación, se describe en detalle el proceso seguido para alcanzar este resultado, destacando las fases principales, los desafíos enfrentados y las soluciones adoptadas en cada etapa.

Cada uno de los estudios parciales y visualizaciones presentados a continuación ha requerido aplicar técnicas específicas de procesamiento de datos, adaptadas al tipo de análisis que se buscaba destacar. Entre las más utilizadas se encuentran la ordenación para facilitar la agrupación visual según principios de la Gestalt, la normalización y el escalado para representar variables en rangos comparables, la correlación para explorar relaciones entre variables como duración y número de paquetes, o la combinación de información para sintetizar datos a nivel de grupo (por tipo de ataque, por IP, etc.). En el código puede consultarse el detalle de cómo se aplican estas técnicas en cada caso, y en esta sección a continuación, resaltaré únicamente los aspectos más relevantes a nivel visual y analítico.

Paso 1: Experimentación con Visualizaciones Iniciales

Con los datos preparados, el siguiente reto fue explorar cómo representar visualmente la información de forma clara y útil. Esta etapa fue especialmente iterativa y demandó probar diferentes tipos de gráficos en **Jupyter Notebook** usando librerías como Matplotlib, Seaborn y finalmente Altair.

Al principio, probé gráficos muy básicos: histogramas de conteo, gráficos de barras simples y diagramas de dispersión estáticos. Sin embargo, estas visualizaciones no permitían un análisis dinámico ni facilitaban la comparación interactiva entre tráfico normal y malicioso.

Por ello, di un salto a visualizaciones más interactivas con Altair, que permite filtros y selecciones dinámicas, esenciales para el tipo de análisis que quería desarrollar. Fue aquí donde empecé a construir los primeros módulos del *dashboard*, utilizando

streamlit, con gráficos que mostraban la cantidad de conexiones y la distribución de tipos de ataques.

Cada visualización fue objeto de pruebas y mejoras sucesivas: ajustar colores para mejorar la diferenciación, cambiar escalas para evitar distorsiones y modificar etiquetas para lograr una lectura más intuitiva.

Rationale para la elección de Streamlit junto con Altair:

Para el desarrollo de la aplicación *dashboard* se optó por utilizar **Streamlit** como framework principal junto con Altair. Esta decisión se fundamenta en varios aspectos técnicos y prácticos que contribuyen a cumplir con los requisitos del proyecto y optimizar la experiencia de usuario:

1. **Facilidad de implementación y rápida prototipación:** Streamlit permite crear aplicaciones web interactivas en Python con muy pocas líneas de código, lo que agiliza enormemente el desarrollo y permite iterar rápidamente sobre diferentes visualizaciones y componentes. Esta característica fue crucial durante el proceso de exploración y refinamiento del *dashboard*.
2. **Interactividad nativa y sencilla integración con Altair:** Streamlit ofrece soporte nativo para integrar gráficos Altair, lo que facilita la creación de visualizaciones dinámicas e interactivas, como filtros, selecciones y actualizaciones en tiempo real. Altair, basado en la gramática de gráficos Vega-Lite, permite construir gráficos declarativos complejos y visualmente atractivos, ideales para analizar grandes volúmenes de datos de manera clara.
3. **Aplicación autocontenida y fácil despliegue:** Una de las condiciones del proyecto era que la aplicación fuese autocontenida y ejecutable sin errores. Streamlit genera un entorno aislado y sencillo de desplegar, compatible con distintos sistemas operativos y entornos virtuales de Python. Además, facilita su publicación en internet mediante servicios como Streamlit Cloud o Heroku, aumentando la accesibilidad y la posibilidad de compartir el trabajo.
4. **Potencial para incorporar elementos interactivos avanzados:** Streamlit soporta fácilmente la inclusión de controles como selectores, sliders, y botones, que enriquecen la experiencia del usuario y permiten explorar el *dashboard* desde múltiples perspectivas. Esta capacidad contribuye a una mejor comprensión de los patrones del tráfico de red normal y malicioso.
5. **Comunidad activa y documentación robusta:** Ambas herramientas cuentan con una comunidad amplia y recursos actualizados, lo que facilita

la resolución de problemas y la incorporación de mejores prácticas durante el desarrollo.

Se consideraron otras alternativas como **Dash** (de Plotly) o **Panel** (de HoloViz), pero fueron descartadas por la mayor complejidad en la integración directa con Altair y la necesidad de configuraciones adicionales para la gestión de visualizaciones interactivas. Además, Streamlit ofrecía una curva de aprendizaje más rápida, un despliegue más sencillo y una sintaxis más limpia para el desarrollo ágil del dashboard, cumpliendo así con los requisitos de autocontenibilidad y facilidad de uso que se exigían para la aplicación.

Paso 2: Experimentación con las visualizaciones para la visión general

Durante el desarrollo de la visión general del *dashboard*, se llevó a cabo un proceso iterativo de experimentación con diferentes tipos de visualizaciones. El objetivo era encontrar una forma efectiva y clara de representar la información más relevante sobre el volumen y la naturaleza del tráfico de red, diferenciando entre tráfico normal y malicioso.

Inicialmente, se probaron distintos tipos de gráficos, como líneas temporales, gráficos de áreas y combinaciones de gráficos apilados, con el objetivo de representar la evolución del tráfico por días. Sin embargo, debido a que el dataset sólo contiene datos correspondientes a tres días, estos gráficos resultaban visualmente pobres o poco informativos, con distribuciones dispares y huecos vacíos que dificultaban la interpretación.

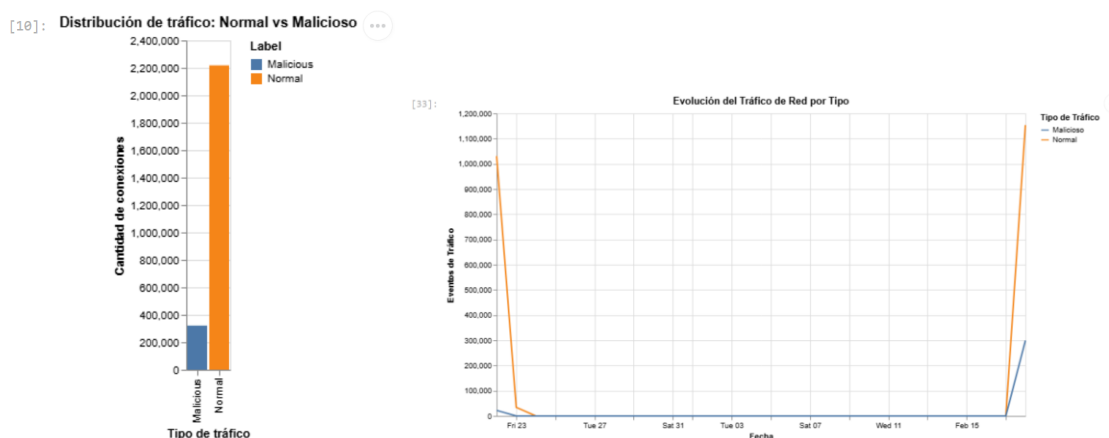


Imagen 1 Pruebas realizadas y finalmente descartadas

Tras varias pruebas, se optó por un enfoque más directo y comprensible:

- **Tabla de conteo de conexiones por día y tipo (normal o malicioso):** Se eligió una tabla simple, que permite visualizar de forma clara y precisa

cuántas conexiones se registraron en cada día, diferenciadas por tipo. Esta opción resultó ser la más efectiva dada la escasa cantidad de días representados en el dataset.

- **Gráfico de tarta para representar el porcentaje de tráfico malicioso vs normal:** Este tipo de visualización ofrece una imagen clara y visualmente intuitiva del volumen relativo de tráfico malicioso frente al legítimo, lo que ayuda rápidamente a entender la proporción de ataques en el conjunto de datos.
- **Gráfico de barras horizontales con el conteo por categorías de ataque:** ordenado de mayor a menor frecuencia, orientado horizontalmente para mejorar la legibilidad de las etiquetas. Esta elección permite identificar de un vistazo qué categorías de ataque son más comunes en el tráfico malicioso.
- **Top 10 de protocolos, servicios y direcciones IP más frecuentes:** Para mostrar los elementos más relevantes del tráfico de red, se optó también por gráficos de barras horizontales. Además, se añadió un filtro interactivo que permite al usuario seleccionar si desea visualizar los datos correspondientes al tráfico normal o malicioso. Esta capacidad de interacción añade valor al análisis al facilitar comparaciones rápidas y centradas. Se valoró la inclusión de un desplegable para esta selección, pero dado el reducido número de opciones, se consideró más cómodo y accesible utilizar botones radiales (*radio buttons*).

Paso 3: Experimentación con las visualizaciones para la Composición del Tráfico Malicioso

Al analizar el tráfico malicioso, me encontré con la complejidad de que cada tipo de ataque presenta características propias. Esto supuso un nuevo desafío: diseñar visualizaciones que permitieran visualizar estas características de forma rápida y comprensible.

Nuevamente, experimenté con diversos diagramas: gráficos de tarta para composiciones y gráficos de barras apiladas. Las primeras versiones hechas con gráficos de barras apiladas fueron demasiado difíciles de interpretar, ya que no permitían comparar la cantidad de los valores en proporciones. Por lo que iteré hacia versiones más limpias con gráficos de tarta. Realicé varias pruebas, creando filtros por característica (por ejemplo, ver todos los ataques según protocolo) y filtros por ataque.

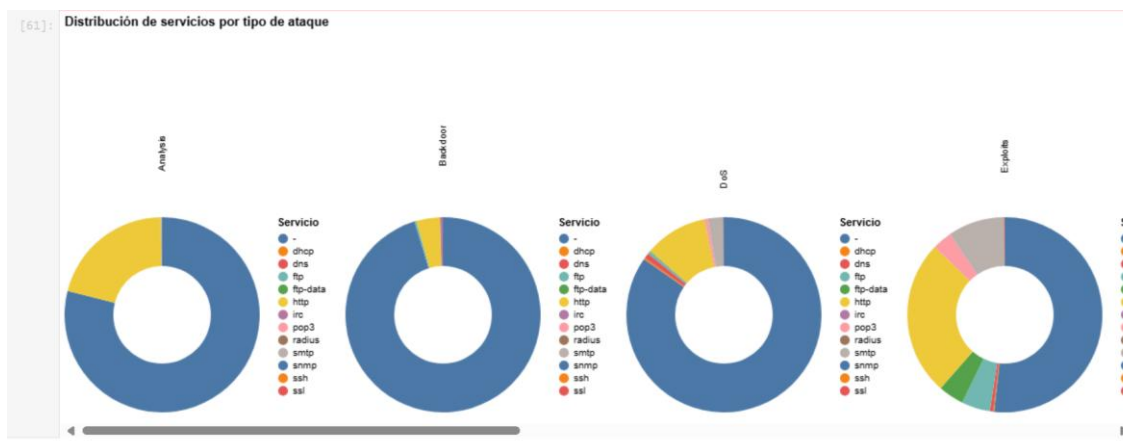


Imagen 2 Ejemplo de prueba que finalmente fue descartada

Finalmente, concluí que la opción más útil y comprensible era permitir al usuario seleccionar un tipo de ataque y mostrar directamente su distribución interna en términos de protocolos, servicios, estados, etc. Esta decisión **facilita la exploración de la composición dentro de cada tipo de ataque**, ayudando a identificar características sin distracciones, y reduce la carga cognitiva al evitar mostrar información excesiva al mismo tiempo. Además, favorece una exploración más dirigida, permitiendo al usuario partir de una pregunta concreta (“¿cómo se comporta un ataque DoS?”) y obtener una respuesta clara. La elección de gráficos de torta refuerza esta estrategia, ya que representan proporciones de forma inmediata y visualmente atractiva, destacando con claridad la composición interna de cada ataque.

Paso 4: Análisis de los Efectos en el Rendimiento de la Red

Aquí las visualizaciones se centraron en distribuciones y relaciones entre bytes enviados/recibidos, duración de conexiones y cantidad paquetes.

Para esta parte, empleé diagramas de caja y gráficos de dispersión con el objetivo de comparar el rendimiento de la red entre tráfico normal y malicioso en términos de bytes enviados/recibidos, duración de las conexiones y cantidad de paquetes.

Inicialmente, estas visualizaciones resultaban difíciles de interpretar, especialmente en los diagramas de dispersión, donde la superposición de puntos dificultaba la lectura. Para mejorar la claridad, el uso de colores diferenciados según el tipo de tráfico fue clave, ya que permitió identificar patrones visuales de manera más rápida e intuitiva, viendo que el tráfico malicioso en general tiene duraciones más largas y más cantidad de paquetes.

Además, implementé un muestreo aleatorio de los datos para reducir el tiempo de carga de los gráficos y hacerlos más interactivos, aunque esto supone una pérdida

de exhaustividad. En una aplicación en producción, una solución más robusta sería emplear técnicas de agregación dinámica o renderizado progresivo para preservar la información sin comprometer el rendimiento.

Por otro lado, en el diagrama de caja de los bytes enviados y recibidos, se observó una dispersión extremadamente alta, lo que hace que la caja (que representa el rango intercuartílico) quede visualmente comprimida en la parte inferior, dando la impresión de que los datos están más concentrados de lo que realmente están. Esto pone en evidencia la cantidad de *outliers* que tienen esta variable.

Esta etapa también requirió iterar varias veces para elegir las métricas y gráficos que reflejaran con precisión las diferencias relevantes sin perder el foco en la usabilidad.

Paso 5: Análisis del Impacto de cada Tipo de Ataque

En esta etapa, el objetivo fue analizar cómo afectan los distintos tipos de ataques al comportamiento de la red, tanto en términos de sus actores (IPs de origen y destino) como en métricas de volumen y calidad del tráfico.

Para identificar las IPs más relevantes en el tráfico malicioso, se crearon gráficos de barras apiladas que muestran las direcciones IP de origen y destino con mayor número de conexiones, segmentadas por categoría de ataque. Estos gráficos permiten visualizar de forma clara los focos principales de emisión y recepción de tráfico hostil, lo cual resulta útil para priorizar acciones defensivas. Y también la tipología de ataque que es capaz de emitir o recibir cada IP. Se aplicó un umbral mínimo de ocurrencias para evitar ruido por valores poco frecuentes y simplificar hacia los casos más representativos.

Además, se desarrollaron visualizaciones que comparan el comportamiento medio de cada tipo de ataque a través de indicadores como el número medio de paquetes medios enviados y recibidos (Spkts y Dpkts), así como la pérdida media de paquetes en origen y destino (sloss y dloss). Para ello, se optó por representaciones lado a lado con escala compartida, lo que facilita la comparación directa y ayuda a detectar ataques que generan un mayor volumen o degradación de tráfico.

En cuanto a utilizar promedios por tipo de ataque, esta decisión se tomó tras experimentar inicialmente con diagramas de caja, los cuales resultaron difíciles de interpretar en este contexto.

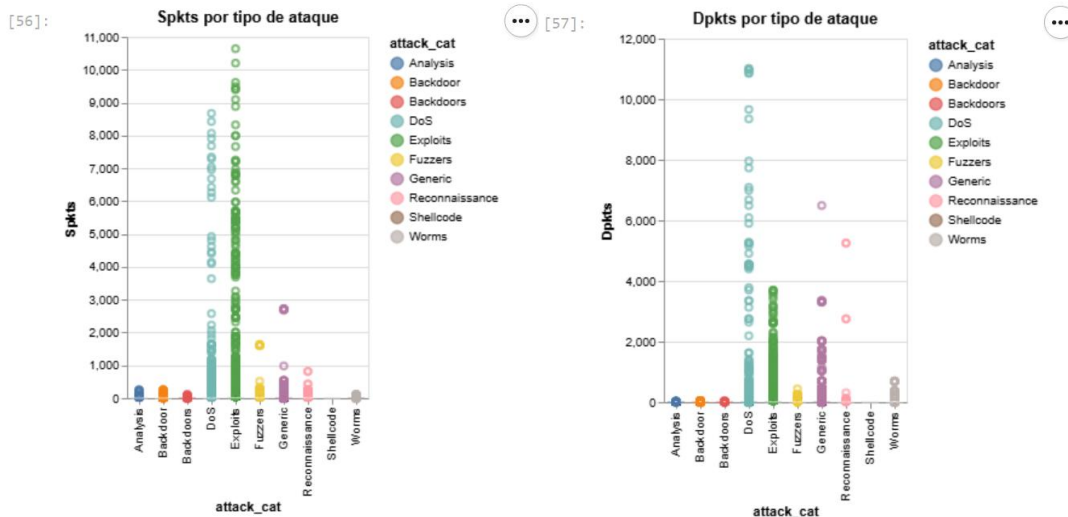


Imagen 3 Diagramas de cajas descartadas

La dispersión dificultaba transmitir una idea clara de las diferencias entre ataques. En cambio, las medias permitieron una comparación directa y más accesible para el usuario, facilitando la interpretación general del comportamiento típico de cada ataque.

Visualización (integración de los resultados parciales)

Enfoque general

Se ha seguido una estructura modular y progresiva, dividida en cuatro pestañas principales que reflejan el proceso de análisis y descubrimiento de información. Cada apartado se construye sobre los hallazgos anteriores, guiando al usuario desde una visión general del tráfico hasta un análisis detallado del impacto de cada tipo de ataque.

Estructura narrativa del dashboard (storytelling)

El *storytelling* visual es uno de los pilares fundamentales del *dashboard*. La estructura por pestañas no es solo organizativa, sino que está cuidadosamente diseñada para contar una historia de forma lógica y creciente en complejidad:

1. **Visión General del Tráfico de Red:** La primera pestaña ofrece una panorámica global del tráfico. Aquí se muestran los volúmenes diarios, los protocolos y servicios más comunes, y las IPs más activas. El objetivo es proporcionar un contexto inicial, detectar tendencias de tráfico y establecer una línea base desde la que partir. Las visualizaciones permiten filtrar por tipo de tráfico (normal, malicioso o ambos), lo que ayuda a hacer comparaciones rápidas. Este enfoque permite al usuario hacerse una primera idea del "estado de la red" antes de profundizar.
2. **Composición del Tráfico Malicioso:** Una vez identificada la proporción y distribución de tráfico malicioso, la segunda pestaña se centra exclusivamente en su análisis. Aquí se descompone por tipo de ataque, mostrando las características de conexión asociadas (puertos, servicios, estados de sesión, etc.). Se ha integrado un filtro por tipo de ataque para observar su comportamiento individual. Esta segmentación permite estudiar las firmas típicas de cada categoría y sirve como base para futuros modelos de detección.
3. **Efectos en el Rendimiento de la Red:** En esta parte se evalúan los efectos cuantitativos que tienen los ataques sobre el rendimiento de la red, comparando métricas como los bytes enviados/recibidos, duración de conexión y número de paquetes entre tráfico normal y malicioso. Esta pestaña combina tanto *boxplots* como gráficos de dispersión, ajustados para mejorar su claridad con técnicas como el muestreo aleatorio y el uso de color. A partir de estas visualizaciones, se pueden detectar ataques que

sobresalen por generar conexiones inusualmente largas o con una cantidad desproporcionada de paquetes.

4. **Impacto de cada Tipo de Ataque:** Finalmente, se analiza en detalle el efecto de cada tipo de ataque por separado. Se incluyen gráficos de IPs origen y destino, así como métricas agregadas de paquetes y pérdida de datos. Esta sección responde a preguntas como: ¿qué ataques causan más congestión? ¿cuáles se asocian a mayores tasas de pérdida? La finalidad es entender no solo el comportamiento de cada ataque, sino también su posible impacto en la red y en los recursos del sistema.

Integración de resultados parciales

Cada sección del *dashboard* no solo presenta información nueva, sino que integra visualmente los hallazgos anteriores para ofrecer una visión coherente y enriquecida:

- El botón radial de filtrado dinámico por tipo de tráfico normal o malicioso en la primera pestaña permite reutilizar o actualizar la información en los mismos gráficos, y usarlos para distintos datos, aumentando la reutilización del código y facilitando la comparación directa.
- En las secciones centradas en el tráfico malicioso, los resultados ya interpretados (por ejemplo, los gráficos de tarta de servicios o protocolos más utilizados) se utilizan como marco para entender mejor la distribución dentro de cada tipo de ataque.
- En la parte de rendimiento, los *boxplots* y los gráficos de dispersión se apoyan mutuamente para representar tanto la distribución estadística como los casos extremos visuales, ayudando a explicar las diferencias observadas previamente entre tráfico normal y malicioso.
- Finalmente, para medir el impacto de los ataques se han utilizado valores promedios por tipo de ataque, que permiten cerrar el análisis con una visión más sintética y comparativa, enfocada en los indicadores de mayor impacto.

Esta integración evita la fragmentación y promueve una narrativa fluida: el usuario no necesita interpretar cada gráfico de forma aislada, sino que cada uno complementa al anterior en una línea lógica de exploración.

Paleta de colores

Para garantizar una experiencia visual coherente, clara y alineada con el dominio de la ciberseguridad, se ha optado por una temática oscura personalizada, tanto a

nivel de la configuración de Streamlit como en los gráficos individuales del *dashboard*.

El archivo de configuración `config.toml` fue ajustado con los siguientes parámetros:

```
[theme]
base="dark"
backgroundColor="#0e1117"
secondaryBackgroundColor="#1c1f26"
textColor="#ffffff"
font="sans serif"
```

Esta configuración establece un **tema visual oscuro personalizado**, con el objetivo de mejorar la legibilidad, reducir la fatiga visual y proporcionar una estética coherente con el contexto de la ciberseguridad, donde muchas veces las amenazas cibernéticas se representan sobre una base o estética de color negro. La opción `base="dark"` activa el modo oscuro, mientras que `backgroundColor` y `secondaryBackgroundColor` definen tonos oscuros específicos para el fondo principal y los elementos secundarios, respectivamente, generando un contraste agradable y profesional.

El `textColor` blanco asegura una buena visibilidad del contenido textual sobre el fondo oscuro, y la fuente `sans serif` ofrece una lectura limpia y moderna. Esta elección contribuye tanto a la usabilidad como a una apariencia técnica y sobria, habitual en herramientas de análisis y entornos de trabajo especializados.

Además, se empleó una codificación de **colores** consistente y semántica **para diferenciar** claramente **los tipos de tráfico analizado** en todos los gráficos interactivos del dashboard:

- Tráfico **Normal** se representa siempre en **verde** (#10b981).
- Tráfico **Malicioso** se representa en **rojo** (#f43f5e).

Este esquema binario, apoyado en la percepción intuitiva del verde como “seguro” y el rojo como “peligroso”, facilita una rápida interpretación visual de los datos sin necesidad de recurrir constantemente a la leyenda. Esta decisión busca minimizar la carga cognitiva del usuario y favorecer la detección inmediata de patrones anómalos, especialmente en entornos donde el tiempo de reacción es crítico.

La coherencia visual se ha mantenido en todas las pestañas del dashboard, reforzando la usabilidad general del sistema y favoreciendo un *storytelling* fluido que permite al usuario pasar de una vista general a análisis más específicos sin perder el contexto visual ni semántico.

Discusión, Conclusiones y posibles mejoras

El desarrollo del presente *dashboard* ha sido una experiencia intensa, marcada por la complejidad inherente al tratamiento de datos de tráfico de red y al reto de representar visualmente información técnica de forma comprensible, atractiva e interactiva.

El proceso ha implicado tanto decisiones analíticas como de diseño visual, con múltiples iteraciones en la búsqueda de un equilibrio entre exhaustividad técnica y claridad comunicativa.

Discusión y Conclusiones

El dashboard desarrollado permite **explorar el tráfico de red en entornos IoT** desde distintas perspectivas, diferenciando de forma clara entre tráfico normal y malicioso, lo cual es esencial para detectar patrones de ataque y anomalías visualmente.

Gracias a su **estructura en pestañas**, el usuario navega de forma lógica desde una visión general, pasando por la composición interna de los ataques, hasta llegar al análisis del impacto en la red por tipo de ataque.

Desde el punto de vista técnico, se realizó un **preprocesamiento exhaustivo del dataset** y una **transformación** adecuada de las variables. El trabajo exploratorio previo en Jupyter Notebook fue clave para garantizar la calidad de los datos.

En lo visual, el uso de **Altair** permitió generar gráficos claros e interactivos. Se aplicó una codificación de colores coherente (verde para tráfico normal, rojo para malicioso), y se configuró un tema oscuro personalizado, acorde al contexto de ciberseguridad y diseñado para **mejorar la legibilidad y reducir la fatiga visual**.

Se cuidó especialmente el **storytelling visual**, organizando el contenido para facilitar una interpretación progresiva y coherente. Cada gráfico fue diseñado con foco en la claridad analítica, evitando representaciones confusas. Por ejemplo, se sustituyeron barras apiladas por gráficos de tarta en la sección de ataques, y se optó por medias en lugar de *boxplots* en el análisis de rendimiento, tras comprobar que la dispersión de los datos dificultaba la interpretación.

Posibles mejoras y ampliaciones futuras

Aunque el resultado final cumple con los objetivos definidos, existen varias líneas de mejora que podrían explorarse para futuras versiones del proyecto:

- **Modularización del código:** Actualmente, todo el código de la aplicación está contenido en un único archivo `dashboard.py`. Para mejorar la

mantenibilidad, escalabilidad y legibilidad del proyecto, sería recomendable dividirlo en varios módulos, por ejemplo:

- `app.py`: archivo principal que ejecuta la aplicación.
- `config.py`: configuración general, colores, constantes, rutas.
- `charts.py`: definición de funciones de visualización (gráficos Altair).
- `preprocessing.py`: funciones para cargar y limpiar datos.
- `utils.py`: funciones auxiliares, como conversores o etiquetas.

Esta estructura facilitaría el trabajo en equipo, la reutilización del código y su despliegue en entornos profesionales.

- **Optimización para grandes volúmenes:** Actualmente se aplica un muestreo aleatorio para ciertos gráficos con el fin de mejorar la velocidad de carga. Para una aplicación en producción, sería deseable incorporar técnicas como *aggregated rendering* o *progressive loading*, que permiten mantener interactividad sin perder detalle.
- **Personalización visual avanzada:** Si se quisiera adaptar el *dashboard* a una organización o producto específico, sería interesante permitir una personalización más profunda de la paleta de colores, logotipo, tipografías o estilos visuales correspondientes a la marca. Streamlit y Altair permiten esta personalización con relativamente poco esfuerzo técnico, y puede contribuir a reforzar la identidad visual de la herramienta.
- **Detección automática de anomalías:** El *dashboard* podría integrarse con sistemas que identifiquen automáticamente comportamientos sospechosos mediante *clustering*, técnicas estadísticas o aprendizaje automático, lo cual añadiría una capa predictiva al análisis descriptivo actual.
- **Personalización de alertas o vistas:** Además de la detección automática, en función del perfil del usuario (por ejemplo, administrador de red vs. analista de seguridad), se podrían crear modos de visualización personalizados con vistas resumidas o centradas en indicadores críticos.
- **Publicación y despliegue online completo:** Aunque el *dashboard* puede ejecutarse localmente sin errores, publicarlo como aplicación web accesible en Streamlit Cloud permitiría su uso compartido en contextos profesionales o educativos.

En resumen, el proyecto ha logrado construir una herramienta visual coherente, funcional y adaptada al dominio de la ciberseguridad en redes IoT. La aplicación es útil no solo como ejercicio académico, sino también como punto de partida para desarrollos más complejos en entornos reales de detección y análisis de amenazas.

Anexo 1: Descripción de variables

Estas son las variables que tenemos en el *dataset* para cada conexión:

Variable	Descripción
srcip	Dirección IP de origen
sport	Puerto de origen
dstip	Dirección IP de destino
dsport	Puerto de destino
proto	Protocolo de la transacción
state	Estado de la conexión y protocolo asociado (p. ej., ACC, FIN, RST, etc.)
dur	Duración total del registro
sbytes	Bytes transferidos desde el origen al destino
dbytes	Bytes transferidos desde el destino al origen
sttl	Valor TTL (time to live) de origen a destino
dttl	Valor TTL de destino a origen
sloss	Paquetes reenviados o perdidos desde el origen
dloss	Paquetes reenviados o perdidos desde el destino
service	Servicio usado (http, ftp, smtp, ssh, dns, etc.)
Sload	Bits por segundo enviados por el origen
Dload	Bits por segundo recibidos en el destino
Spkts	Número de paquetes enviados del origen al destino
Dpkts	Número de paquetes enviados del destino al origen
swin	Valor de ventana de anuncios TCP del origen
dwin	Valor de ventana de anuncios TCP del destino
stcpb	Número base de secuencia TCP del origen
dtcpb	Número base de secuencia TCP del destino
smeansz	Tamaño medio de los paquetes enviados por el origen
dmeansz	Tamaño medio de los paquetes enviados por el destino
trans_depth	Profundidad de conexión en solicitudes/respuestas HTTP
res_bdy_len	Tamaño del contenido real sin comprimir transferido desde el servidor HTTP
Sjit	Jitter (variación en la latencia) del origen en milisegundos
Djit	Jitter del destino en milisegundos
Stime	Tiempo de inicio del registro
Ltime	Tiempo final del registro
Sintpkt	Tiempo entre paquetes del origen
Dintpkt	Tiempo entre paquetes del destino
tcprtt	Tiempo de ida y vuelta para establecer conexión TCP (synack + ackdat)
synack	Tiempo entre SYN y SYN-ACK
ackdat	Tiempo entre SYN-ACK y ACK
is_sm_ips_ports	Vale 1 si la IP y puertos de origen/destino son iguales; si no, vale 0

Variable	Descripción
ct_state_ttl	Número de veces que aparece un estado particular en función de los TTL del origen y destino
ct_flw_http_mthd	Número de flujos que usan métodos HTTP como GET o POST
is_ftp_login	Vale 1 si hay acceso FTP con usuario y contraseña; si no, vale 0
ct_ftp_cmd	Número de flujos que tienen comandos FTP
ct_srv_src	Número de conexiones que comparten el mismo servicio y dirección IP de origen en los últimos 100 registros
ct_srv_dst	Número de conexiones que comparten el mismo servicio y dirección IP de destino en los últimos 100 registros
ct_dst_ltm	Número de conexiones con la misma dirección IP de destino en los últimos 100 registros
ct_src_ltm	Número de conexiones con la misma dirección IP de origen en los últimos 100 registros
ct_src_dport_ltm	Número de conexiones con la misma IP de origen y puerto de destino en los últimos 100 registros
ct_dst_sport_ltm	Número de conexiones con la misma IP de destino y puerto de origen en los últimos 100 registros
ct_dst_src_ltm	Número de conexiones entre una IP de origen y una de destino específicas en los últimos 100 registros
attack_cat	Categoría del ataque (Fuzzers, DoS, Shellcode, Worms, etc.)
Label	Etiqueta binaria: 0 = tráfico normal, 1 = tráfico malicioso

Anexo 2: Instrucciones de despliegue del *dashboard*

Para ejecutar el *dashboard* de análisis de tráfico de red en un entorno local, se deben seguir los siguientes pasos:

Precondiciones

Antes de ejecutar la aplicación, es necesario tener instalado: **Python 3.12.9**

Los siguientes paquetes de Python, que están definidos en el archivo `requirements.txt`:

```
altair==5.5.0
pandas==2.2.3
streamlit==1.44.1
```

Se recomienda utilizar un entorno virtual (como `venv` o `conda`) para evitar conflictos con otras instalaciones de Python.

Para instalar estas dependencias, una vez dentro de la carpeta del proyecto, ejecutar el siguiente comando en la terminal:

```
pip install -r requirements.txt
```

Ejecución del dashboard

Situarse en la carpeta que contiene el archivo `dashboard.py` y ejecutar:

```
streamlit run dashboard.py
```

Esto abrirá automáticamente el dashboard en el navegador predeterminado, normalmente en la dirección: <http://localhost:8501>. Si la aplicación no se abre automáticamente, se puede acceder manualmente desde el navegador utilizando la URL anterior.

Notas adicionales:

- La estructura de carpetas debe mantenerse tal como se proporciona, especialmente si se han definido rutas relativas a archivos CSV o configuraciones.
- Para personalizar los estilos (colores, tema, fuente), se puede modificar el archivo `.streamlit/config.toml`.